

Motion Planning and Collision Avoidance in CoppeliaSim Using OMPL

An Industrial Manipulation Case Study

Ioannis Antonakis

12th January 2026

Originally developed as part of the MSc Electronics and Robotics programme at Kingston University London.

Table of Contents

1.	Introduction	2
2.	Simulation Environment and Scenario Design.....	2
2.1	Simulation Platform and Tools.....	2
2.2	Workspace Layout and Physical Environment.....	3
2.3	Scenario Description	3
2.4	Scene Complexity and Design Rationale.....	4
3.	Robot Control Architecture	5
3.1	Robot Selection and Features	5
3.2	Control Structure and Task Definition	5
3.3	Continuous Operation, Recovery, and Reset Logic	6
4.	Motion Planning and Obstacle Avoidance Strategy	6
4.1	Kinematic Control and Hybrid Dynamics Handling.....	6
4.2	Inverse Kinematics and End-Effector Control.....	7
4.3	Configuration Validation and Collision Handling	7
4.4	Sampling-Based Motion Planning Strategy	7
5.	Simulation Results and Redundancy Exploitation	8
5.1	Planner Performance Evaluation	8
5.2	Exit Placement with Frame Constraint	10
5.3	Shelf Placement with Local Obstacles	11
5.4	Relocated Bin Placement	12
6.	HRI Application - Flexible Manufacturing Systems.....	12
7.	Conclusion and Future Work	13
	References	14
	Appendix A.....	14
	Appendix B.....	15

1. Introduction

Robotic motion planning is a fundamental capability for autonomous and collaborative robotic systems operating in complex, shared environments. In industrial contexts, robots are increasingly required to perform manipulation tasks while adapting to spatial constraints, environmental changes, and safety requirements, particularly when deployed alongside human operators. Simulation-based development provides a safe and efficient platform for designing, testing, and validating such systems before real-world deployment.

This report presents the design and implementation of a robot motion planning scenario developed in CoppeliaSim, using a 7-degree-of-freedom Sawyer robotic manipulator. The simulation focuses on collision-free pick-and-place operations within a constrained workspace that includes conveyors, shelving structures, containers, and additional obstacles. Motion planning is achieved through the integration of inverse kinematics and sampling-based planners from the OMPL library, allowing the robot to dynamically compute feasible joint trajectories for each task execution.

Multiple planning algorithms were evaluated to assess their suitability for confined environments, and a planning strategy was selected based on reliability, motion quality, and robustness to environmental changes. The system exploits kinematic redundancy to avoid obstacles, reach target poses with varying orientations, and adapt its motion in response to workspace reconfiguration.

The scenario is motivated by Human–Robot Interaction in flexible manufacturing systems, where collaborative robots must safely and efficiently adapt to evolving layouts, task requirements, and shared workspaces. The report details the simulation environment, motion planning strategy, experimental demonstrations, and a discussion of how the proposed system supports adaptable and human-centred industrial automation.

2. Simulation Environment and Scenario Design

2.1 Simulation Platform and Tools

The simulation was developed using CoppeliaSim (Educational version), selected for its integrated support for inverse kinematics (IK), collision detection, and sampling-based motion planning through the simIK and simOMPL libraries. Robot motion is generated entirely through scripted kinematic control using Lua, with dynamics responsiveness disabled, in accordance with the coursework requirements.

Sampling-based motion planning is employed to generate collision-free joint-space trajectories between task-defined end-effector poses. This approach allows the robot to adapt its configuration when obstacles are repositioned, thereby supporting the demonstration of redundancy exploitation and robustness in cluttered environments.

2.2 Workspace Layout and Physical Environment

The overall scene layout of the simulation is shown in *Figure 1*, which illustrates a compact industrial-style pick-and-place workspace with a single robotic manipulator. The robot is positioned at the centre of the scene and is surrounded by multiple fixed structures that intentionally intrude into its workspace, creating a non-trivial motion planning problem.

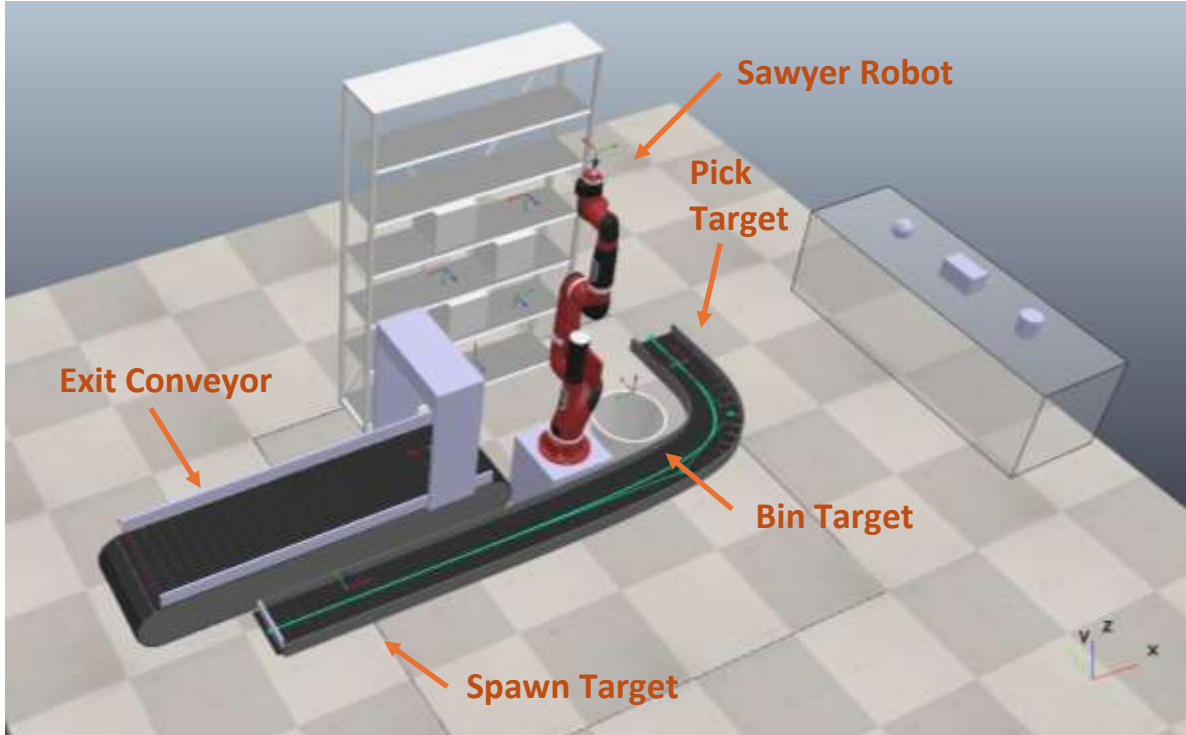


Figure 1 - Idle scene layout containing all elements of the simulation

Left to the robot is a rack with multiple shelves placed within reach. Four shelves contain a predefined target pose used for sequential object placement. On the right is an L-shaped conveyor system delivering objects towards the robot. The curved conveyor being close to the robot introduces additional collision constraints.

A bin is also positioned between the robot and the incoming conveyor. This includes a target pose and is used for object dropping tasks. Behind the robot, a secondary exit conveyor is placed, onto which selected objects are deposited and transported out of the scene.

2.3 Scenario Description

In this scenario, objects enter the scene by spawning onto the conveyor system (thin L-shaped conveyor) one at a time, which are then transferred to the pickup zone where a proximity sensor detects them as normally happens in a real-life scenario. Once an object is detected, the conveyor belt is halted, and the robot moves to pick-up the object. After the robot picks the object, it selects a placement target based on the object type and computes a new path to reach and place the object. Finally, after completing its task the robot returns to its home position, the conveyor system is re-enabled and another object spawns repeating the process. More details will be given in chapter 3.

There are three geometrically distinct object types used in this simulation (see Figure 1), each representing a different handling objective. Rectangular cuboid objects are sequentially placed onto shelf targets, cylindrical objects are deposited onto the exit conveyor, and spherical objects are dropped into a bin. Although the objects are geometrically simple, their differentiated destinations introduce branching task logic and require the robot to approach placement targets from varying directions and orientations.

2.4 Scene Complexity and Design Rationale

The scene is intentionally designed to present a non-trivial motion planning problem without altering task goals or relying on artificial constraints. Fixed structures such as conveyors, shelving, bins, and floor-level obstacles intrude into the robot's reachable workspace, reducing the available configuration space and increasing the likelihood of self-collision or environmental collision during motion execution.

Additional complexity is introduced through fixed obstacles including the boxes placed on the shelves, a frame over the exit conveyor, and rectangle at the floor level, to further restrict the configuration space and enforce collision-aware reconfiguration. These elements restrict approach directions and force the robot to exploit its kinematic redundancy to reach identical target poses under different spatial constraints.

Rather than relying on abstract geometric obstacles, the scene emphasizes clutter commonly found in industrial environments, such as storage structures, material handling equipment, and auxiliary frames. This design choice aligns the simulation with realistic manufacturing conditions while still enabling systematic evaluation of motion planning behaviour.

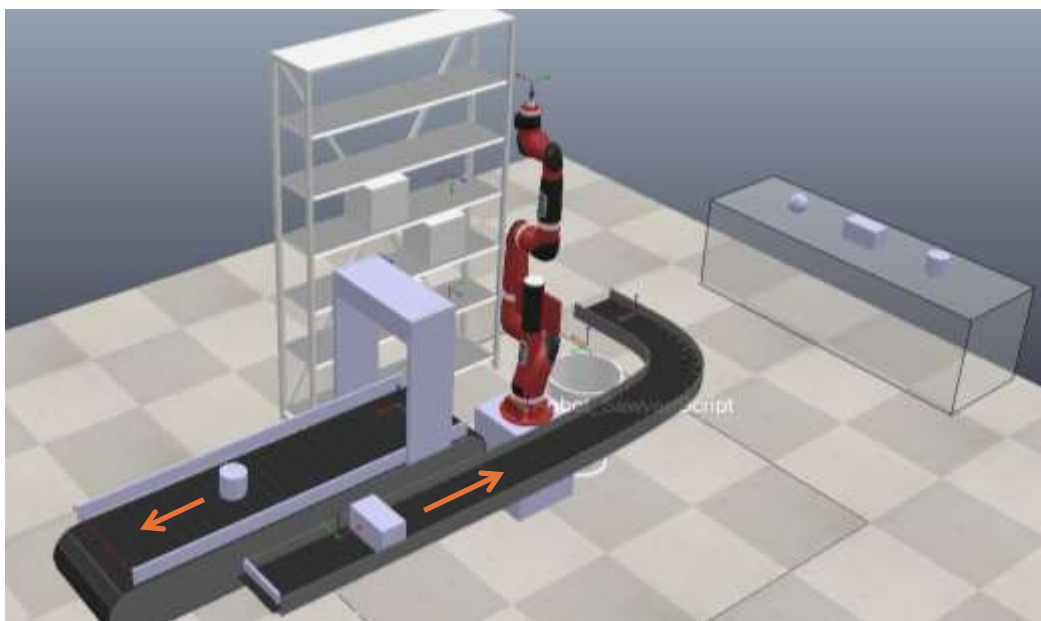


Figure 2 - Objects entering and leaving scene

3. Robot Control Architecture

3.1 Robot Selection and Features

The Sawyer robotic manipulator was selected primarily due to its 7 degrees of freedom (DOF), which exceed the dimensionality of the assigned end-effector task and therefore guarantee kinematic redundancy. This redundancy is essential for achieving obstacle avoidance through reconfiguration while maintaining fixed end-effector poses.

In addition to its kinematic properties, Sawyer was chosen for its large physical workspace, which allows for more nearby obstacles without over constraining its movement, but a platform was still used to elevate the robot and help it reach further. The robot is also designed for collaborative applications, featuring integrated cameras on the base and arm. While these sensors are not used in the current simulation, they provide a plausible foundation for future extensions involving perception-driven or human-aware behaviours.

3.2 Control Structure and Task Definition

For testing purposes, a simple UI pop-up is presented at the start of the simulation, allowing the user to select which object types will be spawned. This avoids the need to modify code between test runs and supports controlled evaluation of different task branches.

The main program is located in a threaded script under the robot hierarchy, which allows running long tasks without blocking the entire simulation. For a more precise and manageable control structure, a finite state machine (FSM) is implemented for handling all robot tasks/operations, allowing for coordinate perception, manipulation, and recovery actions in a deterministic and modular manner.

The steps described in Chapter 2, where the robot tasks take place, are broken down into four main stages. These are defined using the following FSM states:

- **WAITING:** Waiting for object detection at the pickup zone
- **PICKING:** Executing a pickup motion
- **PLACING:** Executing a placement task at the assigned target
- **SPAWNING:** Next object spawns and parts of the system reset

The event-based control strategy allows the system to respond consistently to variable object arrival times and supports reliable operation over extended simulation runs, as will be explained further.

3.3 Continuous Operation, Recovery, and Reset Logic

The system is designed for continuous operation rather than single-cycle execution. After each placement, the robot returns to a predefined home configuration, establishing consistent initial conditions for subsequent motion planning requests. This reset improves planner reliability and reduces sensitivity to accumulated joint deviations.

Once the robot picks an object it only moves to the next stage (PLACING) if successful. From there, the placement is programmatically decided based on the object type, and the robot plans a trajectory to reach that target (see Chapter 4 Motion Planning). If the target placement cannot be achieved due to either failure to find a suitable IK configuration or a collision free path, then the robot will try again and if it fails the second time, it will then try to place the object at a fallback location. The fallback target was chosen to be the exit conveyor, due to having a higher chance of success against the shelves and bin, making a safe recovery. In the unlikely event where this attempt also fails, for example if an obstacle is blocking the exit conveyor target, then the robot will return to its home position and drop the object for a complete recovery allowing the simulation to continue to run.

To support continuous operation and extended testing, automatic cleanup mechanisms are implemented. Shelf-mounted objects are removed once all placement slots are filled, and objects transported on the exit conveyor are deleted once they leave the workspace. These measures prevent uncontrolled scene growth while preserving identical task conditions across repeated cycles.

4. Motion Planning and Obstacle Avoidance Strategy

4.1 Kinematic Control and Hybrid Dynamics Handling

Robot motion in the simulation is generated using purely kinematic control, with dynamics responsiveness disabled for the manipulator. This design choice ensures that joint trajectories produced by inverse kinematics and motion planning are executed deterministically, without interference from physics-based effects such as inertia, contact forces, or joint compliance. As the focus of the coursework is on motion planning and IK configuration, disabling robot dynamics avoids instability and simplifies evaluation.

In contrast, manipulated objects are initially spawned with dynamic properties so that they can be transported realistically and interact with gravity and conveyor motion. Once an object is detected at the pickup zone, its dynamics are disabled before “grasping” which is done programmatically without physics involved. This prevents conflicts that could arise by the physics engine and non-dynamic robot.

After placement, object dynamics are re-enabled, allowing objects to settle naturally on shelves, fall into the bin, or be transported away on the exit conveyor. This hybrid approach reflects common practice in industrial robot simulation and provides a stable yet realistic manipulation pipeline.

4.2 Inverse Kinematics and End-Effector Control

All robot motion is specified in terms of desired end-effector poses rather than individual joint commands. A target dummy is added as the tip of the robot which defines the end-effector pose and helps align the robot with the placement target locations. A pick target is placed over the sensor to ensure both the object and end-effector align, allowing the study to focus on placement complexity rather than grasp planning. All placement targets that differ in orientation from the pickup pose, requiring full 6-DOF pose control rather than simple translational motion.

Inverse kinematics is thus solved using the simIK library, which computes joint configurations that satisfy the pose constraints while respecting joint limits. Due to the Sawyer manipulator having 7DOF, multiple valid IK solutions can exist for a given target pose. This redundancy allows the robot to reach identical end-effector poses through different joint configurations, which is essential for obstacle avoidance in cluttered environments.

Rather than selecting an arbitrary IK solution, all valid configurations are retained for further evaluation and filtering, as described in the following sections.

4.3 Configuration Validation and Collision Handling

To ensure collision-free motion, all candidate joint configurations are validated against the environment before execution. Collision checking is performed using a robot-specific collision collection that includes the full manipulator while excluding irrelevant scene elements. (See *configurationValidationCallback* function, Appendix A.)

This validation mechanism is applied consistently during both inverse kinematics solution generation and motion planning. During IK computation, candidate configurations that result in self-collision or environmental collision are discarded. During motion planning, collision checking is enforced throughout the entire joint-space trajectory.

By integrating collision validation at multiple stages of the planning pipeline, the system prevents infeasible motions from propagating into execution and improves overall robustness in cluttered environments.

4.4 Sampling-Based Motion Planning Strategy

Collision-free trajectories between joint configurations are generated using sampling-based motion planning provided by simOMPL (an API plugin based on the Open Motion Planning Library). For each motion request, a new planning task is created, specifying the robot joint state space, collision pairs, start and goal configurations, and planner-specific parameters.

Motion planning is performed entirely in joint space to exploit the robot's kinematic redundancy and allow reconfiguration around obstacles. The start state is taken from the robot's current joint configuration, while the goal state is selected from valid IK solutions.

Different planners are used depending on task context. The SBL (Single-query Bidirectional Lazy) planner is used for placement motions within cluttered areas of the workspace, as it provides more reliable exploration of constrained regions. RRTConnect is used for pick and return-to-home motions, where the environment is largely unobstructed and faster planning is sufficient. Other OMPL planners were also evaluated during development, including KPIECE1, BKPIECE1, RRTstar, but the previous two proved to be most suitable for the given scenario (see Chapter 5).

To improve motion continuity, the goal configuration is selected as the IK solution closest to the current joint state using a joint-space distance metric (see *selectClosestConfig* function, Appendix B). This reduces unnecessary joint motion and improves repeatability across task cycles by reducing the likelihood of planner failure.

5. Simulation Results and Redundancy Exploitation

5.1 Planner Performance Evaluation

The OMPL planners evaluated during development exhibited noticeably different behaviour when applied to the constrained workspace of the simulation.

SBL demonstrated the most consistent performance, reliably generating collision-free trajectories that reached the desired target poses, even in tightly constrained regions of the workspace. It showed a good balance between planning success and motion quality, making it well suited for place operations in cluttered environments.

RRTConnect also produced reliable solutions but occasionally generated unnecessarily complex joint motions, particularly near obstacles. Despite this, its fast convergence made it suitable for unconstrained motions, such as returning the robot to its home configuration.

KPIECE1 and BKPIECE1 exhibited less consistent behaviour, with occasional failures to reach the correct target pose and, in the case of BKPIECE1, increased planning times. RRT* frequently failed to converge to valid target poses within the available planning time and was therefore unsuitable for this scenario.

These observations, as can be seen in the following figures, motivated the final planner selection described in Chapter 4 and form the basis for the experimental demonstrations presented in the following sections.

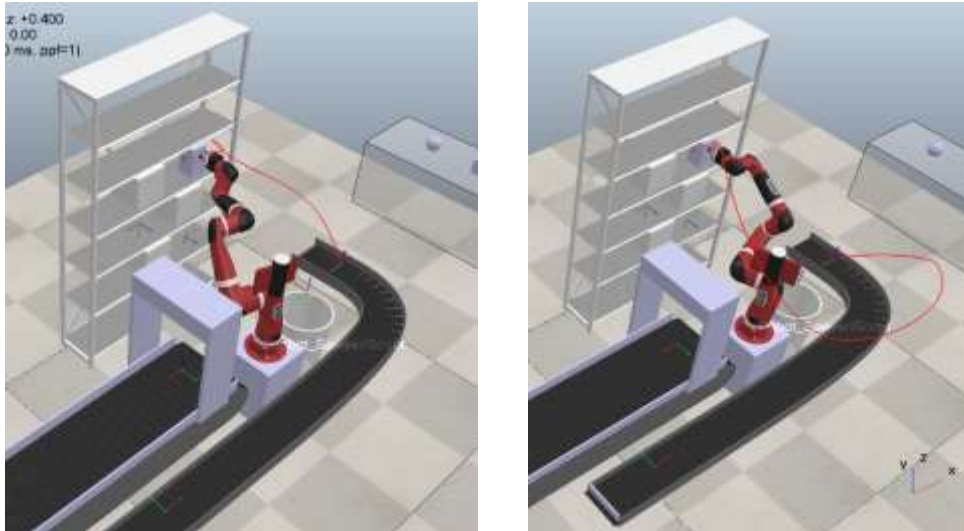


Figure 3 – SBL planner path (left) versus RRTConnect planner path (right). Visualization has been enabled with a red line showing the path drawn during the simulation run.

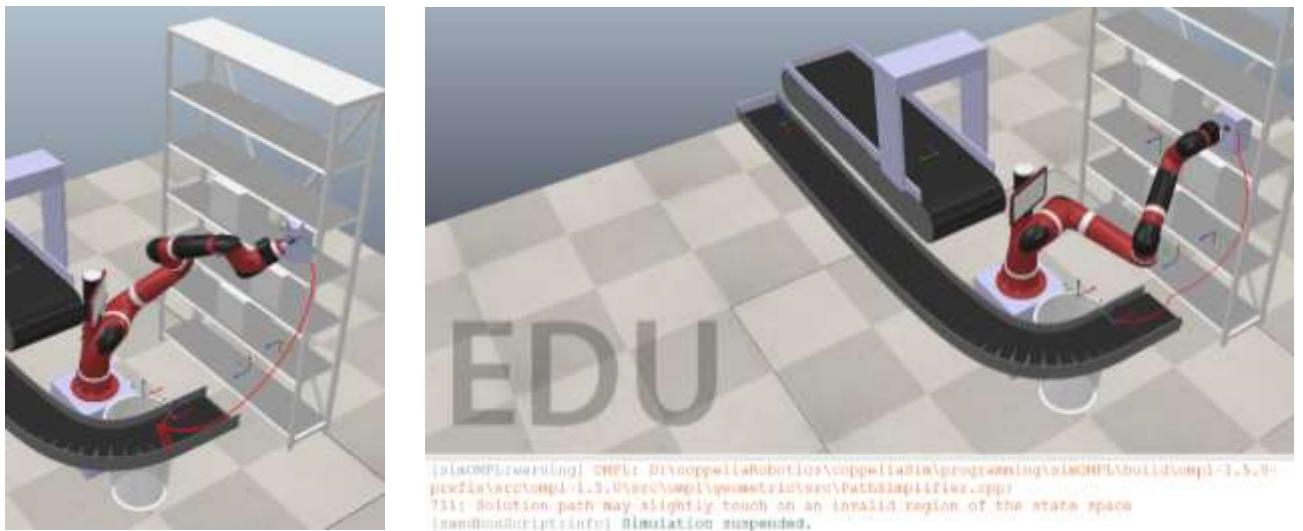


Figure 4 - KPIECE1 planner (left) reaches the target placement but drops object incorrectly (wrong orientation). BKPIECE1 planner (right) achieves the task but with minor collision warning (see console messages).

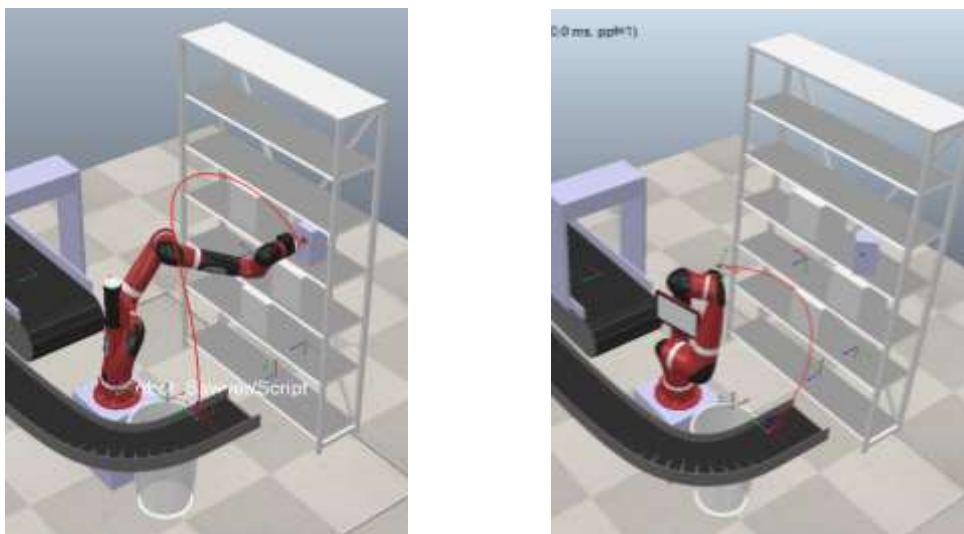


Figure 5 - SBL planner (left) succeeds again on the same task, while RRT* (right) appears to drop object incorrectly without even completing a correct path.

5.2 Exit Placement with Frame Constraint

The rigid frame structure positioned above the exit placement target restricts vertical and lateral clearance during approach of the robot. This represents a typical structure found over industrial conveyors, such as scanning tunnels, gantries, or general support frames.

When the frame is positioned directly above the target, the robot exhibits a distinct change in joint configuration compared to the unconstrained case. While the end-effector reaches the same pose, the arm adopts a more compact posture, with noticeable changes in elbow height and wrist orientation to avoid collisions.

If the frame is moved away from the target location, the constraint is removed and the robot approaches the target using a more direct trajectory with increased freedom, as shown in the figures below. These results demonstrate how kinematic redundancy enables the robot to adapt its configuration while satisfying identical task constraints.

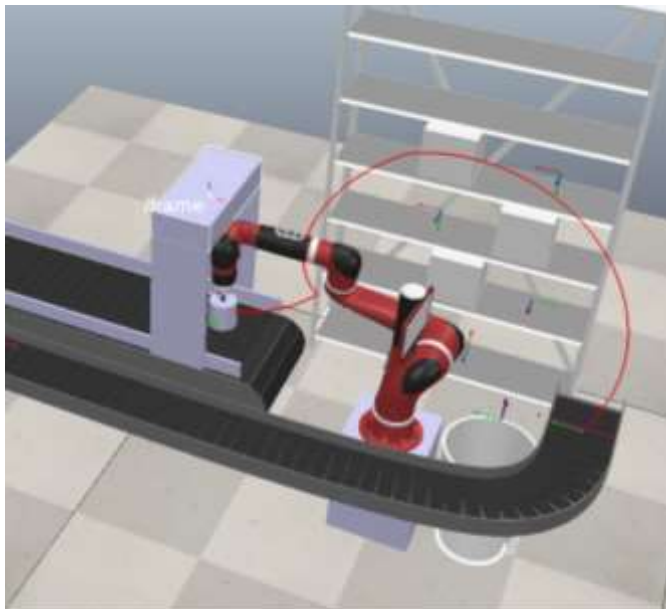
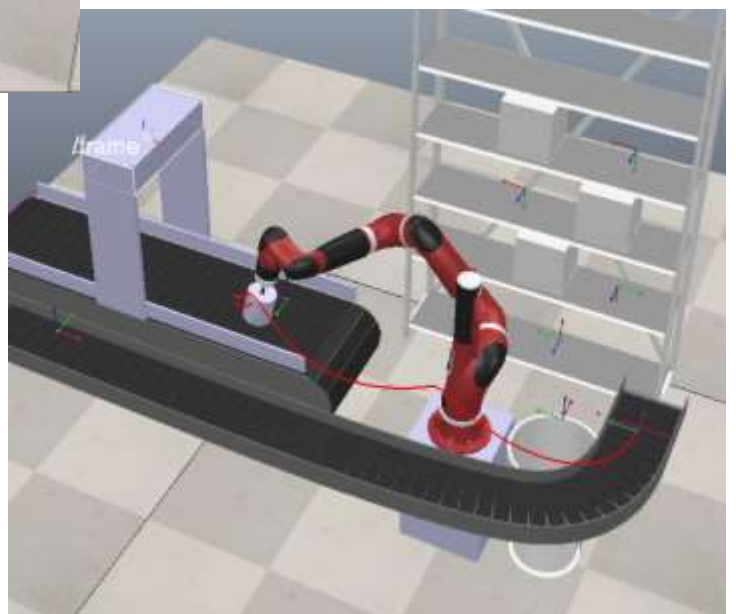


Figure 6 (Left) - Cylinder placement with frame limiting the robot path. The planner needs to find a more constrained path to reach the target.

Figure 7 (Right) - Cylinder placement with no obstructions. The robot follows a less constrained smoother path.



5.3 Shelf Placement with Local Obstacles

A second experiment focuses on the boxes placed on the shelving rack, adjacent to selected shelf targets, representing partially occupied storage locations. These are rigid obstacles that can be moved before or during the simulation to further constraint the motion path, reflecting realistic scenarios where surrounding workspace conditions may change due to human activity during operation.

The shelf structure already constrains motion above and below each target. Introducing these obstacles forces the robot to modify its approach direction. In some cases, the robot performs a larger or more complex motions than would normally be necessary, while in other cases it fails to reach a valid path, triggering the fallback placement described in Chapter 3.

Different box placements result in visibly different trajectories, while the task definition remains unchanged as shown in Figures 8 and 9. This behaviour highlights the robot's ability to exploit redundancy to resolve competing constraints in a cluttered environment.

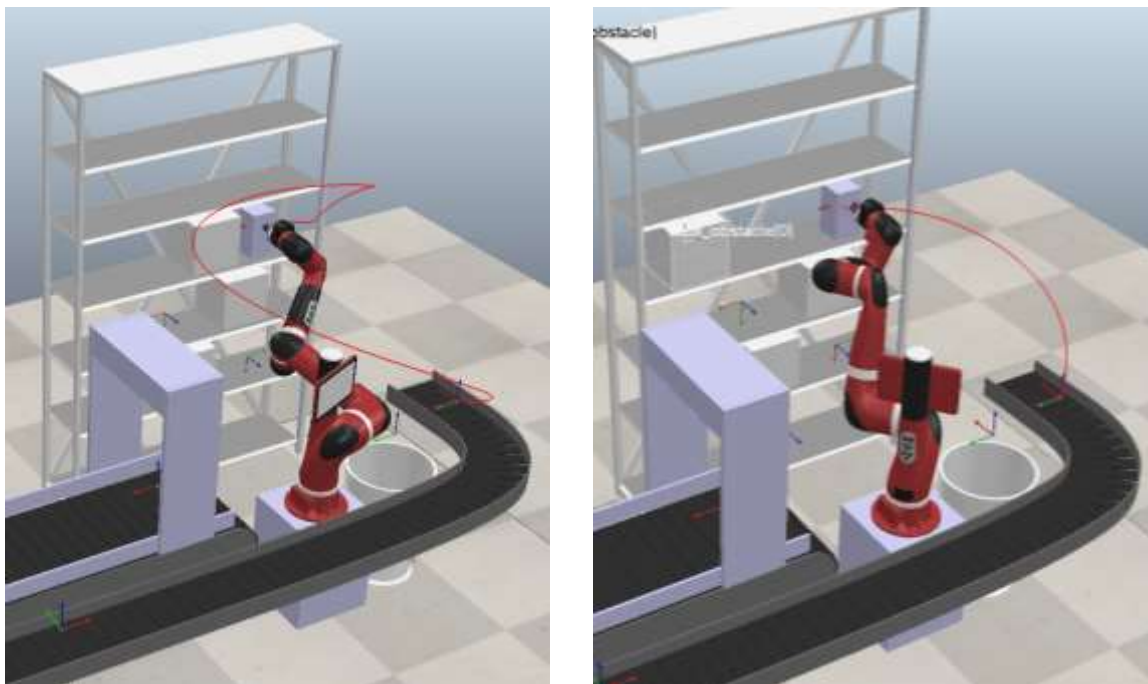


Figure 8 (Left) - Box is too close to placement target, so robot follows a more complex trajectory to reach target.

Figure 9 (Right) - Box is now moved so robot follows a simpler path without obstruction.

5.4 Relocated Bin Placement

As an additional demonstration, the bin and its associated target were repositioned within the workspace. Although the target pose changes in this case, the robot must still plan collision-free paths around fixed obstacles such as the shelving rack and conveyor systems.

This experiment illustrates the continued necessity of motion planning especially when task locations vary, due to the persistent presence of environmental constraints.

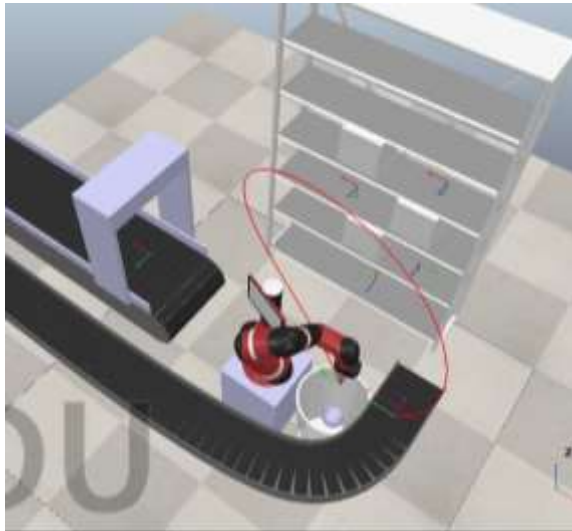


Figure 10 (Left) - Sphere object is placed in the default bin location.

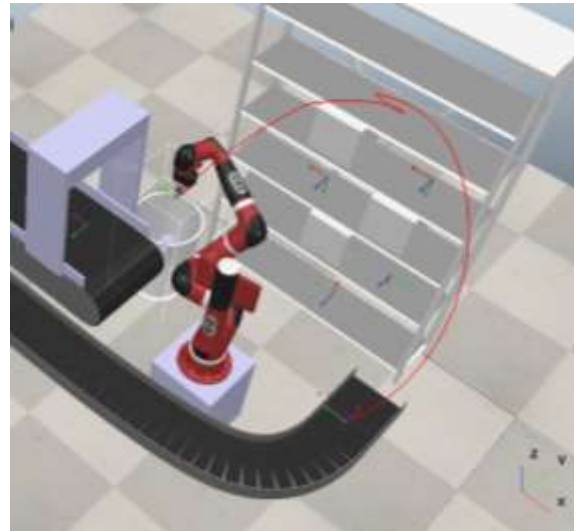


Figure 11 (Right) - Bin repositioned so new path is computed and successfully reached.

6. HRI Application - Flexible Manufacturing Systems

The developed simulation represents a collaborative robotic cell within a Flexible Manufacturing System (FMS). Such systems are designed to accommodate frequent changes in product type, production volume, and workspace layout, prioritising adaptability and efficient reconfiguration over fixed, product-specific production lines (Hayes, 2025; VDL Steelweld, 2025).

In realistic FMS environments, human operators routinely interact with the workspace by repositioning shelves, placing containers, or introducing auxiliary structures to support new products. These activities alter the spatial constraints of the robot's workspace without changing the underlying task requirements. Material handling and spatial organisation are therefore critical aspects of flexible manufacturing performance (Aized, 2010).

This behaviour is reflected in the proposed simulation. Fixed obstacles such as shelves, overhead frames, and containers represent infrastructure elements that may be repositioned by human workers between or during production runs. The robot task remains defined as a set of desired end-effector poses, while the surrounding constraints are modified. The robot must recompute collision-free trajectories without redefining the task itself.

The use of a redundant 7-degree-of-freedom manipulator enables this adaptability. Multiple joint configurations can satisfy the same end-effector pose, allowing the robot to modify its posture in response to changes introduced by human activity. In the simulation, identical placement tasks are achieved through different joint configurations when obstacles are repositioned, demonstrating effective redundancy exploitation in a shared workspace.

From an HRI perspective, this supports safe and predictable collaboration. Human operators can modify the workspace without requiring robot reprogramming, while the robot continues to perform consistent and repeatable actions. The handling of different object types and destinations further reflects the variability typical of flexible manufacturing systems, where manipulation requirements change depending on the product being introduced (Hayes, 2025).

Overall, the scenario demonstrates how redundancy-aware motion planning can support human–robot collaboration in flexible manufacturing systems by enabling task-level consistency under changing spatial constraints.

7. Conclusion and Future Work

This coursework presented a CoppeliaSim-based motion planning simulation demonstrating collision-free task execution, active redundancy exploitation, and Human–Robot Interaction considerations within a flexible manufacturing context. A redundant 7-degree-of-freedom Sawyer manipulator was used to perform a pick-and-place task defined at the end-effector control point with both positional and orientational constraints, ensuring that redundancy was functionally required rather than incidental.

Sampling-based motion planning using the simOMPL library enabled reliable trajectory generation in a cluttered workspace. Qualitative evaluation of multiple planners led to the selection of SBL for constrained pick-and-place motions and RRTConnect for less restrictive return-to-home movements. The robot consistently achieved identical task goals through different joint configurations while avoiding fixed and repositionable obstacles, demonstrating effective and observable redundancy exploitation.

From an HRI perspective, the scenario reflects collaborative operation in flexible manufacturing systems, where workspace layouts may change due to human intervention. The robot’s ability to adapt its motion without redefining task goals supports safety, predictability, and operational continuity in human-shared environments.

Future work could extend the system by introducing explicit human models, safety zones, or dynamic obstacle handling to further investigate proxemics, comfort-aware motion planning, and online replanning in collaborative manufacturing scenarios.

References

Aized, T. (2010) *Flexible manufacturing systems*. InTech. Available at: <http://www.intechopen.com/books/future-manufacturing-systems/material-handling-in-flexible-manufacturing-system> (Accessed: 7 January 2026).

Hayes, A. (2025) *Flexible Manufacturing System (FMS): Adaptability & Efficiency in Production*. Investopedia. Available at: <https://www.investopedia.com/terms/f/flexible-manufacturing-system.asp> (Accessed: 4 January 2026).

VDL Steelweld (2025) *The power of flexible manufacturing: adapting production for efficiency and profitability*. Available at: <https://www.vdlsteelweld.com/en/knowledge-hub/the-power-of-flexible-manufacturing-adapting-production-for-efficiency-and-profitability> (Accessed: 6 January 2026).

Appendix A

```
-----
-- BASIC CONFIGURATION FUNCTIONS
-----

-- Get current robot joint configuration
function getConfig()
    local config = {}
    for i = 1, #simJointHandles do
        config[i] = sim.getJointPosition(simJointHandles[i])
    end
    return config
end

-- Set robot joint configuration
function setConfig(config)
    for i = 1, #simJointHandles do
        sim.setJointPosition(simJointHandles[i], config[i])
    end
end

-- Collision-checking callback for IK / OMPL
function configurationValidationCallback(config)
    local tmp = getConfig()
    setConfig(config)
    local collisionFree = sim.checkCollision(robotCollection,
sim.handle_all) == 0
    setConfig(tmp)
    return collisionFree
end
```

Appendix B

```

-----
-- OPTIMIZATION FUNCTIONS
-----

--- Selects the IK configuration closest to the current joint state
-- Minimizes joint-space distance to ensure smooth, consistent motion
-- @param configs table of IK joint configurations
-- @return table selected joint configuration

function selectClosestConfig(configs)

    -- Safety check: no IK solutions available
    if not configs or #configs == 0 then
        print('[selectClosestConfig] No configurations provided')
        return nil
    end

    local current = getConfig()    -- get current configuration
    local bestConfig = configs[1]  -- choose 1st config by default
    local bestIndex = 1            -- store index for reference
    local bestDist = math.huge     -- initialize with a huge number

    -- Assuming 1 or more configs found
    for i = 1, #configs do
        local d = 0
        for j = 1, #current do
            local diff = configs[i][j] - current[j]
            d = d + diff * diff
        end
        if d < bestDist then
            bestDist = d
            bestConfig = configs[i]
            bestIndex = i
        end
    end

    -- Debugging (optional)
    print('[selectClosestConfig] Selected config:', bestIndex)

    return bestConfig
end

```